

Random Forest - Complete Simple Guide

Multiple decision trees, bagging, bootstrap samples, random feature selection, voting, OOB evaluation, hyperparameters, and banking examples

1. What is a random forest?

A random forest is an ensemble model made from many decision trees. Each tree sees a slightly different sample of the data and considers only a random subset of features at each split. Their predictions are combined to make the final result.

Single decision tree	Random forest
One tree makes the decision	Many trees make decisions
Can be easy to visualize	Overall model is harder to visualize
Can change greatly with small data changes	Averaging makes predictions more stable
May overfit when deep	Usually reduces variance and overfitting

Memory aid

One tree is one opinion. A random forest gathers many varied opinions and combines them.

2. Classification and regression

Task	How predictions are combined	Banking example
Classification	Trees vote or average class probabilities	Fraud / Genuine
Regression	Average the numeric predictions	Expected loss amount

How a Random Forest Is Built

Bagging plus random feature selection creates diverse trees

3. Bootstrap sampling

For each tree, the algorithm draws a training sample with replacement. A record may appear more than once, while some records are left out. This is called a bootstrap sample.

Original records	Possible bootstrap sample for Tree 1
A, B, C, D, E	B, D, D, A, C

4. Random feature selection

At each node, a tree considers only a random subset of the available features. This prevents the same strongest feature from dominating every tree and increases diversity.

5. Grow many trees

Each bootstrap sample and feature subset produces a somewhat different tree. Individual trees are commonly grown deeply; the forest controls variance through averaging rather than relying only on pruning.

6. Combine predictions

Tree	Fraud prediction
Tree 1	Fraud
Tree 2	Genuine
Tree 3	Fraud
Tree 4	Fraud
Tree 5	Genuine
Final majority vote	Fraud (3 of 5)

Why Random Forest Usually Works Better

Diversity is essential: correlated errors do not cancel well

7. Bagging

Bagging means bootstrap aggregating: train models on bootstrap samples and aggregate their predictions. It mainly reduces variance, making the final prediction more stable than a single high-variance tree.

Bias and variance

Model	Bias	Variance	Typical behavior
Shallow decision tree	Higher	Lower	May underfit
Deep decision tree	Lower	Higher	May overfit
Random forest	Usually moderate/low	Lower than one deep tree	Often generalizes well

8. Out-of-bag evaluation

Because each bootstrap sample leaves out some training records, those out-of-bag records can test that tree. Combining these predictions gives an OOB score, which provides an internal performance estimate without creating another validation set.

Important

OOB evaluation is available when `bootstrap=True` and `oob_score=True`. Keep a final untouched test set for the last unbiased evaluation.

9. Split criteria

Every tree still needs to select splits. Classification forests can use Gini impurity, entropy, or log loss. Entropy measures uncertainty; information gain is the reduction in entropy. The forest adds randomness around this familiar tree-building process.

Random Forest Hyperparameters

The most important controls and their effects

Hyperparameter	Simple meaning	Typical effect
n_estimators	Number of trees	More trees improve stability but use more time and memory
criterion	Split-quality measure	Gini, entropy/log_loss; regression criteria differ
max_depth	Maximum depth of each tree	Lower values reduce complexity
min_samples_split	Records needed before a node can split	Higher values reduce small splits
min_samples_leaf	Minimum records in each leaf	Higher values smooth predictions
max_features	Features considered at each split	Lower values increase tree diversity
bootstrap	Whether to sample with replacement	Required for standard bagging and OOB
oob_score	Calculate out-of-bag score	Internal validation estimate
class_weight	Extra importance for a class	Helps with imbalance
max_samples	Bootstrap sample size per tree	Controls data used by each tree
max_leaf_nodes	Maximum leaves per tree	Limits complexity
min_impurity_decrease	Minimum split improvement	Prevents weak splits
ccp_alpha	Pruning strength for each tree	Higher values prune more
random_state	Seed for reproducibility	Same seed gives repeatable results
n_jobs	CPU workers	-1 uses available processors

Practical starting point

Begin with enough trees for stable results, then tune max_features, max_depth, min_samples_leaf, min_samples_split, class_weight, and possibly max_samples using cross-validation. More trees alone do not correct poor data or data leakage.

Feature Importance and Interpretation

Useful signals, but not proof of causation

10. Impurity-based importance

The model adds up how much each feature reduces impurity across all trees. Features responsible for larger reductions receive higher importance.

11. Permutation importance

Shuffle one feature in validation data and observe how much performance falls. A large drop means the model relied strongly on that feature. This is often easier to interpret against actual predictive performance.

Caution	Why it matters
Correlated features	Importance may be split across similar features
Many possible split points	Impurity importance may favor such variables
Protected attributes or proxies	Importance does not establish fairness or permission to use a feature
High importance	Shows predictive reliance, not cause-and-effect

12. Probability and threshold

A classification forest can average the probabilities from its trees. A threshold then converts the probability into a class. Lowering a fraud threshold may increase recall but also increase false positives. The business cost of each error should guide the threshold.

Three Banking Use Cases

How a forest combines many trees for practical predictions

Use case	Target	Possible features	Key evaluation focus
Loan default	Default / No Default	Credit score, debt ratio, income stability, missed payments	Recall, precision, fairness, calibration
Fraud detection	Fraud / Genuine	Amount, time, device, location, transaction history	Recall, precision, PR-AUC, review workload
Customer churn	Leave / Stay	Usage, complaints, balances, products, tenure	Recall, lift, intervention cost

Example 1 - Loan default

Different trees see different applicants and feature subsets. One tree may emphasize credit score; another may emphasize missed payments. Their combined probability is usually more stable than one tree's rule.

Example 2 - Fraud detection

Rare fraud requires careful class weighting, threshold selection, and evaluation. Accuracy can be misleading because predicting every transaction as genuine may still appear highly accurate.

Example 3 - Customer churn

The forest can capture nonlinear interactions, such as frequent complaints becoming especially important for customers with short tenure and low engagement.

Evaluation, Strengths, and Limitations

A good score must translate into a sound banking decision

13. Evaluation

Need	Useful approach
Classification quality	Confusion matrix, precision, recall, F1, ROC-AUC; PR-AUC for rare positives
Probability quality	Calibration curve and Brier score
Reliable estimate	Cross-validation, OOB score, then final test-set evaluation
Operational impact	Cost of false positives/negatives, latency, workload, fairness, drift

14. Advantages

- Often more accurate and stable than one decision tree.
- Captures nonlinear relationships and interactions.
- Usually needs little feature scaling.
- Handles classification and regression.
- Offers OOB evaluation and feature-importance methods.

15. Limitations

- Harder to explain than one small tree.
- More trees require more memory and computation.
- Can still overfit noisy data if poorly configured.
- Probability estimates may need calibration.
- Feature importance requires careful interpretation.

Decision Tree vs Random Forest

Choose based on interpretability, stability, and performance

Aspect	Decision tree	Random forest
Models	One tree	Many trees
Interpretability	High for a small tree	Lower overall
Stability	Can be unstable	Usually stable
Overfitting risk	Higher when deep	Usually lower
Training/prediction cost	Lower	Higher
Scaling required	Usually no	Usually no
Best use	Clear rules and explanation	Strong general predictive performance

Final memory aid

Random forest = bootstrap samples + random feature subsets + many trees + combined predictions. Its strength comes from diversity and averaging. Tune with validation, evaluate the right business errors, and keep the final test set untouched.